# Übungsblatt 4: Funktionen und Stack

**Mikrocomputertechnik 1**

**Musterlösung**

## Aufgabe 1: Hinweg und Rückweg im Abfüllprogramm (10 Punkte)

Ein Abfüllautomat füllt aus einem Vorratstank je Abfüllung b Liter ab; er begleitet Sie durch das ganze Blatt. Die Funktion `Restmenge` berechnet den Tankinhalt nach zwei Abfüllungen, $a - 2b$. Sie ruft dazu den Helfer `Entnahme` ($a - b$) auf und zieht danach b ein zweites Mal ab. Das Hauptprogramm hält die Chargennummer 77 in s1 und braucht sie nach dem Aufruf wieder.

RV32I ohne C: Jeder Befehl ist 4 Byte lang, `ret` ist der eine Befehl `jalr x0, 0(ra)`. Die Adressen gelten, wenn der Code bei `0x00` beginnt.

```
.text
.globl main
main:                    # Adresse
    addi s1, x0, 77      # 0x00  Chargennummer
    addi a0, x0, 19      # 0x04  Bestand a in Litern
    addi a1, x0, 6       # 0x08  Abfüllmenge b in Litern
    jal  ra, Restmenge   # 0x0C
Ergebnis_pruefen:
    addi a1, a0, 0       # 0x10  Ergebnis nach a1
    addi a0, x0, 1       # 0x14  Dienst 1: Zahl ausgeben
    ecall                # 0x18
    addi a0, x0, 10      # 0x1C  Dienst 10: Ende
    ecall                # 0x20
Restmenge:
    addi sp, sp, -16     # 0x24  Frame reservieren
    sw   ra, 0(sp)       # 0x28  Rückweg nach main sichern
    sw   s1, 4(sp)       # 0x2C  Chargennummer sichern
    addi s1, a1, 0       # 0x30  b aufbewahren
    jal  ra, Entnahme    # 0x34  a0 = a - b
    sub  a0, a0, s1      # 0x38  a0 = a - 2b
    lw   s1, 4(sp)       # 0x3C  Chargennummer laden
    lw   ra, 0(sp)       # 0x40  Rückweg laden
    addi sp, sp, 16      # 0x44  Frame freigeben
    ret                  # 0x48
Entnahme:
    sub  a0, a0, a1      # 0x4C
    ret                  # 0x50
```

1. Ergänzen Sie nächsten PC und ra nach dem Befehl. Jede Zeile startet mit dem angegebenen ra. Die erste Zeile ist ein Beispiel. (4 P)

| Adresse | Befehl | ra vorher | nächster PC | ra danach |
|---|---|---|---|---|
| `0x0C` | `jal ra, Restmenge` | beliebig | `0x24` | `0x10` |
| `0x34` | `jal ra, Entnahme` | `0x10` | | |
| `0x50` | `ret` | `0x38` | | |
| `0x48` | `ret` | `0x10` | | |
| `0x0C` | `jal x0, Restmenge` | `0x00` | | |

2. Eine Kontrollroutine ruft `Entnahme` zusätzlich mit `jal ra, Entnahme` an Adresse `0x60` auf. Welche Rückkehradresse entsteht? Warum endet `Entnahme` mit `ret` und nicht mit einem festen Sprung nach `0x38`? (2 P)
3. Durch einen Fehler gilt direkt vor dem `ret` bei `0x48` sp = `0x2FF0` statt `0x3000`, außerdem a0 = 7, s1 = 77 und ra = `0x10`. Geben Sie PC, ra, sp und a0 nach dem `ret` an. (2 P)
4. Ein anderer Fehler legt vor dem `ret` bei `0x50` den Wert `0x39` in ra, in einem zweiten Lauf `0x3A`. Wohin springt `ret` jeweils? Welches Ziel ist eine gültige Befehlsadresse? Tipp: `jalr` löscht nur Bit 0. (2 P)

**Musterlösung**

| Adresse | Befehl | ra vorher | nächster PC | ra danach |
|---|---|---|---|---|
| `0x34` | `jal ra, Entnahme` | `0x10` | `0x4C` | `0x38` |
| `0x50` | `ret` | `0x38` | `0x38` | `0x38` |
| `0x48` | `ret` | `0x10` | `0x10` | `0x10` |
| `0x0C` | `jal x0, Restmenge` | `0x00` | `0x24` | `0x00` |

1. Je Zeile 1 P. jal schreibt den alten PC plus 4 nach rd und springt zum Label. ret springt zum Wert in ra; der Linkwert geht an x0 verloren, ra bleibt gleich. `jal x0` ist `j` und springt ohne Link.
2. `0x64` (1 P). Ein fester Sprung schickte auch diesen Aufrufer nach `0x38`. ret liest dagegen den Rückweg des jeweiligen Aufrufs aus ra (1 P).
3. PC = `0x10`, ra = `0x10`, sp = `0x2FF0`, a0 = 7 (1 P). ret springt nur; der Rückweg stimmt, sp bleibt aber 16 Byte zu tief (1 P).
4. `0x39` = $0011\,1001_2$: Bit 0 wird gelöscht, Ziel `0x38`, durch vier teilbar und zufällig richtig (1 P). `0x3A` = $0011\,1010_2$: Bit 0 ist schon null, Ziel `0x3A`. Es ist nicht durch vier teilbar und daher in unserer Basis ohne C keine gültige Befehlsadresse (1 P).

## Aufgabe 2: Wer muss welchen Wert erhalten? (8 Punkte)

Zusätzlich hält `main` in t2 den Flaschenzähler 8, und `Restmenge` liest den Mindestbestand 5 aus s2.

1. Ergänzen Sie, wer für den Wert zuständig ist und was zu tun ist. Die erste Zeile ist ein Beispiel. (4 P)

| Situation | zuständig | Maßnahme |
|---|---|---|
| `main` braucht s1 = 77 nach dem Aufruf wieder, `Restmenge` überschreibt s1 mit b | `Restmenge` (s-Register, callee-saved) | im Prolog sichern, im Epilog laden |
| `main` braucht t2 = 8 nach `jal ra, Restmenge` wieder | | |
| `Restmenge` braucht ra = `0x10` nach `jal ra, Entnahme` wieder | | |
| `main` übergibt a0 = 19 an `Restmenge` | | |
| `Restmenge` liest s2 = 5, verändert s2 aber nicht | | |

2. In einem Testlauf ist t2 nach dem Aufruf noch 8. Darf `main` deshalb auf die Sicherung verzichten? (1 P)
3. Ein Teammitglied will „alle Register wie beim Eintritt“ zurückgeben. Es sichert im Prolog zusätzlich a0 und lädt es im Epilog zurück. Welche Zahl gibt das Programm dann aus, und welche wäre richtig? (2 P)
4. `Entnahme` ruft niemanden auf. Braucht sie einen Frame? Was ändert sich, wenn sie s3 als Zwischenregister benutzt? (1 P)

**Musterlösung**

| Situation | zuständig | Maßnahme |
|---|---|---|
| t2 = 8 nach dem Aufruf | `main` (caller-saved) | vor dem Aufruf sichern, danach laden |
| ra = `0x10` in `Restmenge` | `Restmenge` als Caller | im Frame sichern, vor `ret` laden |
| a0 = 19 | niemand | keine, a0 trägt das Ergebnis 7 |
| s2 = 5 | `Restmenge` | keine, reines Lesen erhält s2 |

1. Je Zeile 1 P. Statt der Sicherung von t2 ist auch ein s-Register in `main` richtig.
2. Nein. Der Lauf zeigt nur diese Implementierung; der Vertrag erlaubt `Restmenge`, t2 zu verändern (1 P).
3. Ausgabe 19 statt 7 (1 P). a0 ist das Ergebnisregister; das gewünschte neue Ergebnis darf nicht durch das alte Argument ersetzt werden (1 P).
4. Ohne s-Register braucht `Entnahme` keinen Frame. Mit s3 muss sie dessen Eintrittswert sichern und laden, also einen 16-Byte-Frame anlegen (1 P).

## Aufgabe 3: Den Frame von Restmenge planen (10 Punkte)

`Restmenge` sichert ra und s1, je 32 Bit. Beim Eintritt gilt sp = `0x3000` (Lehrwert S). Es gilt die 16-Byte-Ausrichtung der Kurskonvention.

1. Bestimmen Sie Nutzbedarf, Framegröße und Padding sowie sp nach `addi sp, sp, -16`. Tipp: $F = 16\lceil B/16 \rceil$. (2 P)
2. Ergänzen Sie das Speicherbild nach den Befehlen bei `0x28` und `0x2C` aus Aufgabe 1. Die erste Zeile ist ein Beispiel. (3 P)

| Adresse | Offset zu sp | Inhalt (hex) | Bedeutung |
|---|---|---|---|
| `0x2FFC` | 12 | unbekannt | Padding |
| `0x2FF8` | | | |
| `0x2FF4` | | | |
| `0x2FF0` | | | |

3. Geben Sie die Bytes an den Adressen `0x2FF4` bis `0x2FF7` in aufsteigender Reihenfolge an. Der Speicher ist Little Endian. (1 P)
4. Ein Entwurf gibt den Frame zuerst frei und lädt danach: `addi sp, sp, 16`, dann `lw s1, 4(sp)`. Von welcher Adresse lädt der `lw`? Warum liefert er nicht die Chargennummer? (2 P)
5. Eine erweiterte `Entnahme` reserviert während des Aufrufs ebenfalls 16 Byte und sichert s3 bei `4(sp)`. An welcher Adresse liegt dieser Slot? Überschreibt er eine der beiden Sicherungen aus der Tabelle? (2 P)

**Musterlösung**

1. Nutzbedarf $B = 8$ Byte, Frame $F = 16$ Byte, Padding 8 Byte (1 P). sp = `0x3000` − 16 = `0x2FF0` (1 P).

| Adresse | Offset zu sp | Inhalt (hex) | Bedeutung |
|---|---|---|---|
| `0x2FF8` | 8 | unbekannt | Padding |
| `0x2FF4` | 4 | `0x0000004D` | alter s1-Wert 77 (Chargennummer) |
| `0x2FF0` | 0 | `0x00000010` | Rückweg nach `main` |

2. Je Zeile 1 P. Padding wird nicht beschrieben; sein Inhalt bleibt unbekannt, nicht null.
3. `4D 00 00 00`: das niederwertige Byte an der kleinsten Adresse (1 P).
4. sp ist schon `0x3000`, gelesen wird also `0x3004` (1 P). Das liegt oberhalb des eigenen Frames im Bereich des Aufrufers; der gesicherte Wert steht an `0x2FF4`. Erst laden, dann freigeben (1 P).
5. Innerer sp = `0x2FE0`, Slot bei `0x2FE4` (1 P). Keine Überschneidung: Der innere Frame umfasst `0x2FE0`–`0x2FEF`, der von `Restmenge` `0x2FF0`–`0x2FFF` (1 P).

## Aufgabe 4: Das Programm verfolgen (14 Punkte)

Vor dem Befehl bei `0x0C` gilt: s1 = 77, a0 = 19, a1 = 6, sp = `0x3000`.

1. Tragen Sie die Werte direkt nach dem genannten Befehl ein. Zwischen zwei Zeilen können weitere Befehle liegen; führen Sie diese mit aus. Die erste Zeile ist ein Beispiel. (8 P)

| nach | Befehl | nächster PC | a0 | s1 | ra | sp |
|---|---|---|---|---|---|---|
| `0x0C` | `jal ra, Restmenge` | `0x24` | 19 | 77 | `0x10` | `0x3000` |
| `0x24` | `addi sp, sp, -16` | | | | | |
| `0x30` | `addi s1, a1, 0` | | | | | |
| `0x34` | `jal ra, Entnahme` | | | | | |
| `0x4C` | `sub a0, a0, a1` | | | | | |
| `0x50` | `ret` | | | | | |
| `0x38` | `sub a0, a0, s1` | | | | | |
| `0x40` | `lw ra, 0(sp)` | | | | | |
| `0x48` | `ret` | | | | | |

2. Welche Wörter stehen an `0x2FF0` und `0x2FF4`, während `Entnahme` läuft? Welcher Wert steht zur selben Zeit im Register ra? (2 P)
3. Vergleichen Sie bei `Ergebnis_pruefen` a0, s1 und sp mit den Werten vor dem Aufruf. Welche Änderung ist gewollt, welche Werte müssen gleich sein? (2 P)
4. Zwei weitere Aufträge lauten a = 50, b = 12 und a = 10, b = 8. Welche Restmengen gibt das Programm aus? Was bedeutet das negative Ergebnis für den Tank? (2 P)

**Musterlösung**

| nach | Befehl | nächster PC | a0 | s1 | ra | sp |
|---|---|---|---|---|---|---|
| `0x24` | `addi sp, sp, -16` | `0x28` | 19 | 77 | `0x10` | `0x2FF0` |
| `0x30` | `addi s1, a1, 0` | `0x34` | 19 | 6 | `0x10` | `0x2FF0` |
| `0x34` | `jal ra, Entnahme` | `0x4C` | 19 | 6 | `0x38` | `0x2FF0` |
| `0x4C` | `sub a0, a0, a1` | `0x50` | 13 | 6 | `0x38` | `0x2FF0` |
| `0x50` | `ret` | `0x38` | 13 | 6 | `0x38` | `0x2FF0` |
| `0x38` | `sub a0, a0, s1` | `0x3C` | 7 | 6 | `0x38` | `0x2FF0` |
| `0x40` | `lw ra, 0(sp)` | `0x44` | 7 | 77 | `0x10` | `0x2FF0` |
| `0x48` | `ret` | `0x10` | 7 | 77 | `0x10` | `0x3000` |

1. Je Zeile 1 P. In der Zeile `0x40` hat der Befehl bei `0x3C` s1 bereits auf 77 zurückgesetzt.
2. `0x2FF0`: `0x00000010`, `0x2FF4`: `0x0000004D` (1 P). Im Register ra steht `0x38`, der innere Rückweg; der äußere liegt nur noch im Slot (1 P).
3. a0 wechselt von 19 auf das Ergebnis 7; das ist gewollt (1 P). s1 = 77 und sp = `0x3000` müssen wie vor dem Aufruf sein und sind es (1 P).
4. $50 - 2 \cdot 12 = 26$ und $10 - 2 \cdot 8 = -6$ (1 P). –6 heißt: Der Tank reicht nicht für zwei Abfüllungen, es fehlen 6 Liter (1 P).

## Aufgabe 5: Fehlersuche ab Symptom (10 Punkte)

Drei Teams haben `Restmenge` selbst geschrieben. `main` und `Entnahme` sind wie in Aufgabe 1, jeder Fall startet wie in Aufgabe 4.

**Team A** spart den Frame und hebt b in t0 auf:

```
Restmenge:                  # Fassung von Team A
    addi t0, a1, 0          # 0x24  b aufbewahren
    jal  ra, Entnahme       # 0x28  a0 = a - b
    sub  a0, a0, t0         # 0x2C  a0 = a - 2b
    ret                     # 0x30
Entnahme:
    sub  a0, a0, a1         # 0x34
    ret                     # 0x38
```

Symptom: Das Programm gibt nichts aus und endet nicht. Hält man es an, steht der PC bei `0x2C` oder `0x30`, und a0 wird immer kleiner.

1. Welchen Wert hat ra nach dem `jal` bei `0x28`? Wohin springt deshalb das `ret` bei `0x30`? (1 P)
2. Geben Sie a0 nach den ersten drei Ausführungen von `sub a0, a0, t0` an. (1 P)
3. Mit welchen zwei Änderungen wird daraus die Fassung aus Aufgabe 1? Begründen Sie beide mit dem Registervertrag. (2 P)

**Team B** hat im Prolog zwei Zeilen vertauscht: Bei `0x2C` steht `addi s1, a1, 0`, bei `0x30` steht `sw s1, 4(sp)`. Symptom: Die Ausgabe 7 stimmt, aber bei `Ergebnis_pruefen` steht in s1 die 6 statt der Chargennummer 77.

4. Welcher Wert landet im Slot `4(sp)`, an welcher Adresse? (1 P)
5. Warum stimmt das Ergebnis trotzdem? (1 P)
6. Wie lautet die Korrektur? (1 P)

**Team C** gibt im Epilog mit `addi sp, sp, 8` frei. Symptom: `main` ruft `Restmenge` für vier Aufträge nacheinander auf. Alle Ergebnisse stimmen, danach liegt sp 32 Byte unter `0x3000`.

7. Welchen Wert hat sp nach dem ersten Aufruf? Ist er durch 16 teilbar? (1 P)
8. Wie lautet die Korrektur? (1 P)
9. Warum korrigiert das `ret` den Fehler nicht, und warum stört er trotz richtiger Ergebnisse? (1 P)

#### Musterlösung

1. ra = `0x2C`; das `ret` bei `0x30` springt nach `0x2C` statt nach `0x10` (1 P). Der Rückweg nach `main` wurde vom inneren `jal` überschrieben.
2. 7, 1, −5: Jeder Durchlauf zieht noch einmal 6 ab (1 P).
3. ra vor dem inneren Aufruf im Frame sichern und vor dem `ret` laden, denn der innere `jal` überschreibt ra (1 P). b in s1 statt t0 aufheben und s1 sichern: t0 ist caller-saved, `Entnahme` dürfte es verändern. Dass diese `Entnahme` t0 nicht anfasst, ist kein Vertrag (1 P).
4. 6, an `0x2FF4` (1 P).
5. Die Rechnung benutzt s1 = 6 als b. Verletzt ist nur die Erhaltung von s1, nicht das Ergebnis (1 P).
6. Erst `sw s1, 4(sp)`, danach `addi s1, a1, 0`: erst sichern, dann überschreiben (1 P).
7. $\texttt{0x3000} - 16 + 8 = \texttt{0x2FF8}$; nicht durch 16 teilbar, Rest 8 (1 P).
8. `addi sp, sp, 16`: Die Freigabe muss der Reservierung entsprechen (1 P).
9. ret springt nur über ra und verändert sp nicht. Spätere sp-relative Zugriffe von `main` treffen falsche Adressen, und die 16-Byte-Ausrichtung ist verletzt (1 P).

## Aufgabe 6: Ausbaustufe mit zwei Abfüllmengen (8 Punkte)

Der Automat füllt nun zwei verschiedene Mengen ab. `Bilanz` erhält a in a0, b in a1 und c in a2 und liefert $a - b - c$ in a0. Sie ruft `Entnahme` zweimal auf und steht im Programm aus Aufgabe 1 direkt hinter `Entnahme`.

```
Bilanz:                  # Adresse
    addi sp, sp, -16     # 0x54
    sw   ra, 0(sp)       # 0x58
    sw   s1, 4(sp)       # 0x5C
    addi s1, a2, 0       # 0x60  c aufbewahren
    jal  ra, Entnahme    # 0x64  a0 = a - b
    _____________________ # 0x68  (B) für die 2. Entnahme
    jal  ra, Entnahme    # 0x6C  a0 = a - b - c
    _____________________ # 0x70  (C)
    _____________________ # 0x74  (D)
    addi sp, sp, 16      # 0x78
    ret                  # 0x7C
```

1. Warum kopiert der Befehl bei `0x60` c nach s1, statt es in a2 zu lassen? (1 P)
2. Ergänzen Sie die Zeilen (B), (C) und (D). (3 P)
3. Geben Sie a0 für a = 19, b = 6, c = 4 nach der ersten und nach der zweiten Entnahme an. (1 P)
4. Jemand fügt direkt nach dem ersten `jal` (also bei `0x68`) zusätzlich `sw ra, 0(sp)` ein, „damit ra vor dem zweiten Aufruf gesichert ist“. Welche Adresse steht danach im Slot `0(sp)`? Wohin springt das abschließende `ret`? (2 P)
5. `Bilanz` soll zusätzlich s2 und s3 als Arbeitsregister benutzen. Wie groß muss der Frame nun sein, und wie viel Padding bleibt? (1 P)

#### Musterlösung

1. a2 ist caller-saved, `Entnahme` darf a2 verändern. s1 überlebt den Aufruf, muss dafür aber im Prolog gesichert werden (1 P).
2. (B) `addi a1, s1, 0`, (C) `lw s1, 4(sp)`, (D) `lw ra, 0(sp)`. Je 1 P; (C) und (D) dürfen vertauscht sein.
3. 13 und 9 (1 P).
4. Der erste `jal` hat ra = `0x68` gesetzt; dieser Wert überschreibt im Slot den Rückweg nach `main` (1 P). Das abschließende `ret` springt nach `0x68` zurück in `Bilanz` statt zu `main` (1 P). Das eingehende ra wird einmal vor dem ersten inneren Aufruf gesichert, nicht vor jedem.
5. Vier Wörter = 16 Byte Nutzbedarf: Frame 16 Byte, kein Padding (1 P).

## Aufgabe 7: Das Abfüllprogramm in Venus prüfen

1. Öffnen Sie die Kurs-Venus unter thaumicmekanism.github.io/venus und geben Sie das Programm aus Aufgabe 1 im Tab *Editor* ein. Ein Klick auf den Tab *Simulator* übersetzt es. Vergleichen Sie die angezeigten Befehlsadressen mit Aufgabe 1. Lesen Sie den Anfangswert S von sp ab und prüfen Sie, ob er durch 16 teilbar ist.
2. Rechnen Sie die Slotadressen aus Aufgabe 3 mit Ihrem S statt `0x3000` um. Setzen Sie Haltepunkte auf `sub a0, a0, a1` in `Entnahme` und auf `addi a1, a0, 0` bei `Ergebnis_pruefen`, und starten Sie mit *Run*. Prüfen Sie am ersten Haltepunkt a0, s1, ra und sp sowie im Tab *Memory* die beiden Slots.
3. Gehen Sie mit *Step* bis zum zweiten Haltepunkt; *Prev* nimmt einen Schritt zurück. Vergleichen Sie mit Ihrer Tabelle aus Aufgabe 4. Welche Zahl erscheint am Ende in der Konsole?
4. Testen Sie die Aufträge (50, 12), (10, 8) und (0, 0), indem Sie die Konstanten in `main` ändern. Wechseln Sie nach jeder Änderung erneut in den Tab *Simulator*. Notieren Sie Ausgabe, s1 und sp bei `Ergebnis_pruefen`.
5. Bauen Sie nacheinander die Fehler von Team B und Team C aus Aufgabe 5 ein. Sagen Sie s1 beziehungsweise sp bei `Ergebnis_pruefen` voraus, bevor Sie das Programm laufen lassen.
6. Optional: Ergänzen Sie `Bilanz` aus Aufgabe 6 hinter `Entnahme` und rufen Sie sie aus `main` mit a0 = 19, a1 = 6, a2 = 4 auf.

**Bedienhinweise zu Venus.** Laut Venus-Dokumentation setzt ein Klick auf eine Instruktion in der Programmansicht des Simulators einen Haltepunkt; die Zeile wird rot, ein zweiter Klick entfernt ihn. *Run* läuft bis zum Programmende oder zum nächsten Haltepunkt, *Step* führt die markierte Instruktion aus, *Prev* nimmt die letzte zurück, *Reset* stellt den Anfangszustand her. Im Tab *Registers* ist das zuletzt geänderte Register hervorgehoben. Im Tab *Memory* springen Sie über das Adressfeld zu einer Adresse; angezeigt werden einzelne Bytes. Unter *Display Settings* wählen Sie *Hex* oder *Decimal*. Bezeichnungen können in der Kursdistribution abweichen; notieren Sie Abweichungen im Protokoll.

### Musterlösung

Modellvorhersagen; Adressen, S und Anzeigen der eigenen Venus-Version gehören ins Protokoll.

Beginnt der Code bei `0x00000000`, der Voreinstellung laut Venus-Dokumentation, stimmen die Adressen mit Aufgabe 1 überein. Die Dokumentation nennt `0x7FFFFFF0` als Beginn des Stacksegments; dieser Wert ist durch 16 teilbar. Mit diesem S liegen die Slots bei `0x7FFFFFE0` (ra = `0x00000010`) und `0x7FFFFFE4` (s1 = 77 = `0x0000004D`).

| Haltepunkt | a0 | s1 | ra | sp |
|---|---|---|---|---|
| vor `sub a0, a0, a1` | 19 | 6 | `0x38` | S − 16 |
| bei `Ergebnis_pruefen` | 7 | 77 | `0x10` | S |

Die Konsole zeigt 7. Die Aufträge ergeben 26, −6 und 0, jeweils mit s1 = 77 und sp = S. In der Hex-Anzeige erscheint −6 als `0xFFFFFFFA`. Mit dem Fehler von Team B ist s1 = 6, mit dem Fehler von Team C ist sp = S − 8.

Optional: Mit dem folgenden Anfang von `main` gibt das Programm 9 aus; alle folgenden Adressen verschieben sich um 4 Byte.

```
main:
    addi s1, x0, 77
    addi a0, x0, 19
    addi a1, x0, 6
    addi a2, x0, 4
    jal  ra, Bilanz
```